

Lecture: 0011

# C To MIPS Translation

## Two Philosophies for a CPU's Language

*The instruction set is the contract between hardware and software —  
and there are two great ways to write it.*

**CSA222: Modern Computer Architecture**

# From Instructions to Programs

*L11 taught us to encode single instructions. Now we express whole programs.*



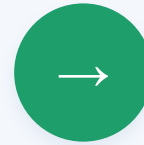
## We can encode

R, I, J formats — any single MIPS instruction, in bits (L11).



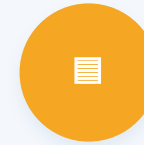
## But programs have structure

if/else, loops, functions — none of which exist as single instructions.



## The translation patterns

Each C construct maps to a fixed idiom of branches and jumps.



## Functions need the stack

Calling conventions and stack frames make reusable code possible.

*The magic of a compiler is just a set of these patterns applied over and over. Today, you become the compiler.*

# Today's Roadmap

*Three patterns, building from a simple if to a recursive function.*

1

## Conditionals

if / else via beq, bne, and the slt idiom for ordering.



2

## Loops

while and for — condition tests and branch-back patterns.



3

## Functions

The calling convention and stack frames — leaf and recursive.

**The skill: see a C construct, recall its MIPS idiom. Once you know the handful of patterns, any program translates.**

# The Translation Mindset

*High-level code is structured; assembly is flat. Translation means flattening structure into labels and branches.*



## Variables → registers

Each active C variable lives in a register (\$t/\$s).  
The compiler assigns them.



## Structure → labels + branches

if, else, loop bodies become labelled blocks;  
control flow becomes jumps between them.



## Conditions → compare + branch

A test like  $a > b$  becomes 'set a flag, then branch on it' — usually `slt` then `bne`.



## Reverse the condition

if (cond) do X → often 'if NOT cond, skip X'. You  
branch AWAY on the opposite test.

*Keep 'reverse the condition' in mind — it's the trick behind almost every if and loop translation today.*

# 1. Conditionals

**if and if/else become compare-and-branch idioms — beq and bne for equality, slt for ordering.**

# The Branch Toolkit

*MIPS keeps it minimal: two branches and one set-less-than. Everything else is built from these.*

**beq \$a,\$b,L**

Branch to L if \$a == \$b  
(branch if equal).

**bne \$a,\$b,L**

Branch to L if \$a != \$b  
(branch if not equal).

**slt \$d,\$a,\$b**

Set \$d = 1 if \$a < \$b, else 0  
(set-less-than).

**j L**

Unconditional jump to  
label L.

*Notice: there's no 'branch if less-than' instruction. You build it: slt to set a flag, then bne with \$zero.*

# Translating a Simple if

*The core trick: branch AWAY on the opposite condition, skipping the body.*

C SOURCE

```
if (a == b) {  
    c = a + b;  
}  
// continue...
```

```
# MIPS assumption of variable ($t0=a, $t1=b, $t2=c)  
  
    bne $t0, $t1, Skip  
    add $t2, $t0, $t1  
Skip:  
    # continue...
```

*C says 'if EQUAL, do the body'. MIPS says 'if NOT equal (bne), skip the body'. Reversed condition, same result.*

# Translating if / else

*Two labelled blocks and an unconditional jump to skip the else after the if.*

## C SOURCE

```
if (a == b) {  
    c = 1;  
} else {  
    c = 0;  
}
```

## MIPS

```
    bne $t0, $t1, Else  
    li  $t2, 1      # c = 1  
    j   Done  
Else:  
    li  $t2, 0      # c = 0  
Done:
```

*The j Done is essential — without it, the if-block would fall straight through into the else-block.*

# Ordering: the slt Idiom

*For < > <= >= there's no direct branch — you set a flag with slt, then branch on it.*

```
C:  if (a < b) ...  
  
    if (a < b) {  
        // body  
    }
```

MIPS

```
    slt  $t3, $t0, $t1  # t3 = (a<b)  
    beq  $t3, $zero, Skip  
    # body (runs when a<b)
```

Skip:

```
# t3=1 means a<b → beq $zero  
# is false → body runs.
```

*slt sets \$t3 to 1 if a<b. Then 'beq \$t3,\$zero,Skip' skips the body when the flag is 0 (i.e. a≥b).*

# Worked: max of Two

*if ( $a > b$ )  $c = a$ ; else  $c = b$ ; — a full, runnable translation.*

## C SOURCE

```
if (a > b)
    c = a;
else
    c = b;
```

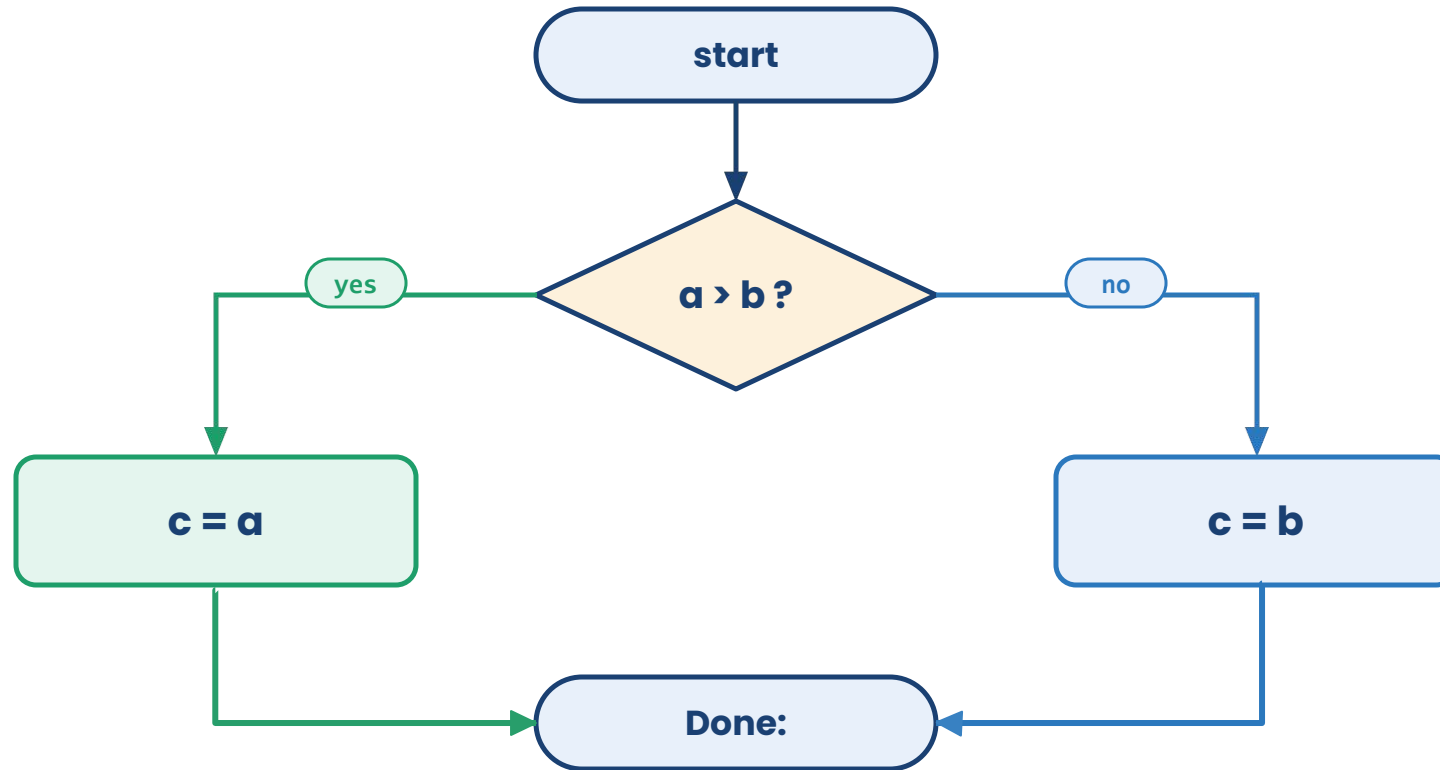
## MIPS (\$t0=a, \$t1=b, \$t2=c)

```
    slt  $t3, $t1, $t0    # t3 = (b < a) = (a > b)
    beq  $t3, $zero, Else
    move $t2, $t0          # c = a
    j     Done
Else:
    move $t2, $t1          # c = b
Done:
```

*Trick:  $a > b$  is the same as  $b < a$  — so we use `slt $t3,$t1,$t0`. Flip the operands to turn  $>$  into  $<$ .*

# Control Flow, Visualised

*The same max example as a flowchart — see how branches carve the two paths.*



*Every if/else is this shape: a test that splits into two blocks that merge back. Branches are the arrows.*

## 2. Loops

**while and for are just conditionals with a branch back to the top — a test, a body, and a jump.**

# The while Loop

*Test at the top; run the body; jump back. Exit when the condition fails.*

## C SOURCE

```
while (i < n) {  
    sum += i;  
    i++;  
}
```

# MIPS assumption (\$t0=i, \$t1=n, \$s0=sum)

Loop:

```
    blt  $t1, $t0, End  
    add  $s0, $s0, $t0    # sum+=i  
    addi $t0, $t0, 1      # i++  
    j    Loop
```

End:

*Pattern: Loop label → test (slt+beq to exit) → body → j back to Loop → End label. Memorise this shape.*

# The for Loop

*A for is a while with the init and increment made explicit — same skeleton underneath.*

## C SOURCE

```
for (i = 0; i < 10; i++) {  
    sum += i;  
}
```

## MIPS (\$t0=i, \$s0=sum)

```
li    $t0, 0           # i = 0  
Loop:  
slti  $t2, $t0, 10     # i<10 ?  
beq   $t2, $zero, End  
add   $s0, $s0, $t0     # sum+=i  
addi  $t0, $t0, 1       # i++  
j     Loop  
End:
```

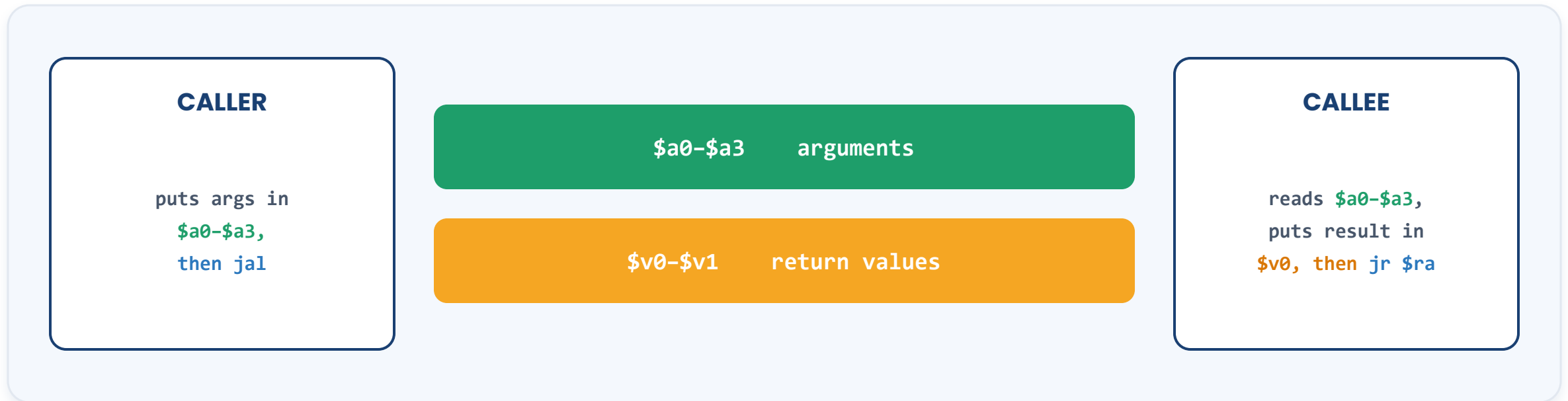
*Map the four for-parts: init before the label, condition at the top, body in the middle, increment before the jump-back.*

# 3. The Calling Convention

A shared agreement on how functions pass arguments, return values, and preserve registers — so any code can call any code.

# Arguments & Return Values

*Four argument registers in, two return registers out — the standard flow of a call.*



*More than 4 arguments? The extras go on the stack. Return values beyond two are rare usually just \$v0.*

**\$a0 - \$a3 = four argument slots.**  
**\$v0 - \$v1 = return slots. This is the single most-used convention.**

# jal and jr \$ra: Call & Return

*Two instructions make a function call work — one to go, one to come back.*

## jal Label

jump and link

- Saves the return address (PC+4) into \$ra.
- Then jumps to Label (the function).
- 'Link' = remember where to return to.
- This is how you CALL a function.

## jr \$ra

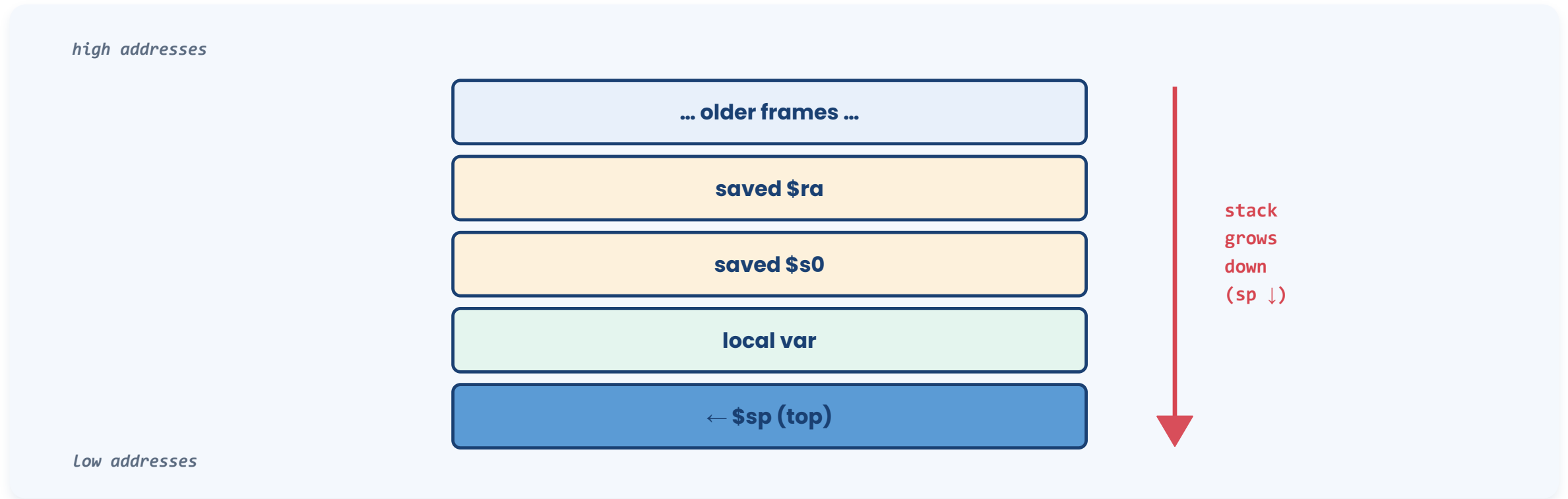
jump register

- Jumps to the address held in \$ra.
- That's the instruction after the original jal.
- So execution resumes at the caller.
- This is how you RETURN from a function.

*Exam favourite: jal stores PC+4 into \$ra AND jumps. jr \$ra reads that address back. Together = call/return.*

# The Stack Grows Downward

*In MIPS, pushing DECREASES  $\$sp$  — the stack grows toward lower addresses.*



*So 'addi  $\$sp$ ,  $\$sp$ , -8' pushes 8 bytes (makes room); 'addi  $\$sp$ ,  $\$sp$ , 8' pops them.  
Down to push, up to pop.*

# Worked: Recursive Factorial

*The classic — each call saves its own \$ra and \$a0 on the stack.*

## C SOURCE

```
int fact(int n) {
    if (n <= 1)
        return 1;
    return n * fact(n-1);
}
```

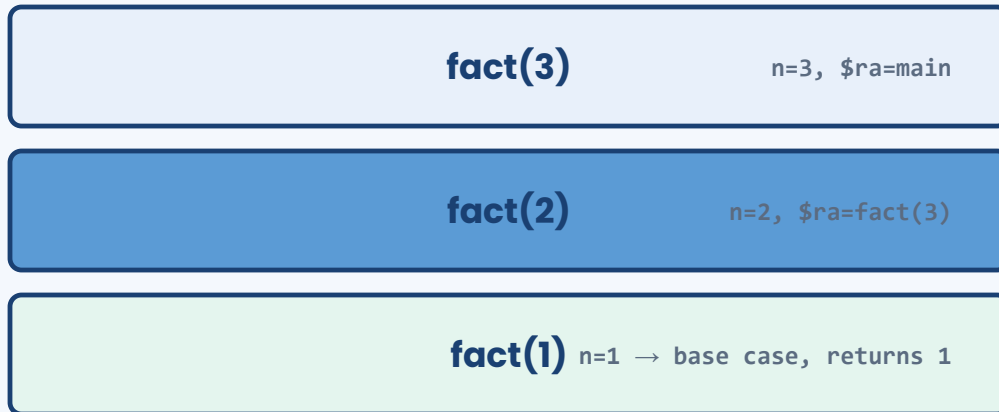
```
main:
# Read n and call fact
li $v0, 5 # read int
syscall
move $a0, $v0 # $a0 = input n
jal fact # call factorial
move $s0, $v0 # save result
li $v0, 10 # exit
syscall

# -----
fact:
# 1. Prologue: Allocate stack & save $ra, $a0
addi $sp, $sp, -8
sw $ra, 4($sp) # save return address
sw $a0, 0($sp) # save current n
# 2. Base Case: Check if n <= 1
li $t0, 1
ble $a0, $t0, base_case
# 3. Recursive Step: Compute fact(n - 1)
addi $a0, $a0, -1 # n - 1
jal fact # recursive call
# 4. Combine Results
lw $a0, 0($sp) # restore original n
mul $v0, $a0, $v0 # n * fact(n - 1)
j fact_exit
base_case:
li $v0, 1 # return 1
fact_exit:
# 5. Epilogue: Restore $ra & stack pointer
lw $ra, 4($sp) # restore $ra
addi $sp, $sp, 8 # deallocate frame
jr $ra # return
```

*Each recursive level gets its own stack frame holding its \$ra and n — that's how recursion 'remembers' where it was.*

# How the Stack Grows: **fact(3)**

*Watch each call push a frame, then unwind as they return.*



↓ *calls push down*

## THE UNWIND

**fact(1) returns 1**

**fact(2) =  $2 \times 1 = 2$**

**fact(3) =  $3 \times 2 = 6$**

As each returns, it pops its frame, restores its saved `n` and `$ra`, multiplies, and returns to its caller.

*3 calls → 3 frames on the stack at peak, then unwound one by one.*

# How the Stack Grows: fact(3)

Watch each call push a frame, then unwind as they return.

**fact(3)**

Frame 1 (Caller)

n=3, \$ra=main

**fact(2)**

Frame 2 (Recursive)

n=2, \$ra=fact(3)

**fact(1)**

BASE CASE REACHED

n=1 → returns 1

↓ Calls push frames downward onto stack

## STEP 2 THE UNWIND (RETURNING)

1. base case: **fact(1)** returns 1
2. unwind 2nd: **fact(2)** = 2 × 1 = 2
3. unwind top: **fact(3)** = 3 × 2 = 6

As each call returns, it **pops its frame**, restores saved n and \$ra, multiplies the result, and returns to caller.

**3 calls → 3 frames on stack at peak, then unwound one by one.**

# The Full Function Template

*Every non-leaf function follows this five-part skeleton — memorise it.*

## 1 Prologue

Make stack room, save \$ra and any \$s registers you'll use.

## 2 Body

Do the work — arguments in \$a0–\$a3, locals in \$t/\$s registers.

## 3 Calls

Use jal for sub-calls; restore any caller-saved values you need after.

## 4 Result

Place the return value in \$v0.

## 5 Epilogue

Restore \$ra and \$s registers, free stack room, then jr \$ra.

# Common Mistakes to Avoid

*The slips that cost marks — and break your programs.*



## Not reversing the condition

if (cond) body → branch AWAY on !cond. Branching on cond itself inverts your logic.



## Using blt / bgt directly

Base MIPS has no blt. Use slt then beq/bne with \$zero.



## Forgetting to save \$ra

A non-leaf function that doesn't save \$ra can never return — jal overwrites it.



## Unbalanced stack

Every addi \$sp,-N must be matched by addi \$sp,+N. Leftover offsets corrupt the caller.



## Wrong array scaling

A[i] is at base + i×4 for words. Forgetting the ×4 (sll by 2) reads the wrong memory.



## Clobbering \$s without saving

If you use an \$s register, you MUST save & restore it — the caller trusts it survives.

# Key Takeaways

*Five sentences — from C constructs to running assembly.*

- 01 if/else translates to compare-and-branch: use beq/bne, and branch AWAY on the reversed condition.
- 02 Ordering (< > <= >=) uses the slt idiom — set a flag with slt, then branch on it with beq/bne + \$zero.
- 03 Loops are a label, a condition test at the top, a body, and a j back — while and for share this skeleton.
- 04 The calling convention: arguments in \$a0–\$a3, returns in \$v0–\$v1, return address in \$ra (set by jal, used by jr).
- 05 Non-leaf and recursive functions build a stack frame: push \$ra & saved regs (stack grows down), pop before jr \$ra.

# Practice & Homework

*Translate control flow, loops, and a recursive function.*

**01**

## if/else

Translate `if (a == b) c = 1; else c = 0;` to MIPS. Show your labels.

**02**

## Loop Sum

Translate `int sum(int n){int s=0; for(i=1;i<=n;i++) s+=i; return s;}` to MIPS.

**03**

## Recursion

Translate a recursive factorial to MIPS with a proper stack frame (save `$ra` & `$a0`).

### EXIT TICKET

1. Where are function arguments passed? Where are return values placed?
2. What exactly does `jal` do, in two steps?
3. Why must a non-leaf function save `$ra`, but a leaf function need not?

# Rapid-Fire Recap

*Three questions — shout the answers.*

**Q1**

**Return values go in which registers?**

`$v0 / $v1`

**Q2**

**jal stores the return address in...**

`$ra`

**Q3**

**The MIPS stack grows in which direction?**

`downward`

*All three instant? You can hand-compile C to MIPS — and Module 4 builds the CPU that runs it.*

END OF L12 • MODULE 3 COMPLETE

# You are the compiler now.

NEXT — MODULE 4, L13

**The Fetch-Decode-Execute Cycle — we finally BUILD the CPU datapath that turns these instructions into action.**